

OpenMP Tutorial

@yukim0359

July 10, 2025

Table of Contents

1. What is OpenMP?
2. Representative Syntax
3. Loop Scheduling
4. Parallelization Methods Other Than for-loops
 - sections Construct
 - task Construct
5. Variable Attributes
6. Considerations on Thread Count
7. Appendix

1. What is OpenMP?
2. Representative Syntax
3. Loop Scheduling
4. Parallelization Methods Other Than for-loops
 - sections Construct
 - task Construct
5. Variable Attributes
6. Considerations on Thread Count
7. Appendix

What is OpenMP?

- **OpenMP** is an API for parallelizing programs on shared-memory parallel computers
 - MP stands for *Multi-Processing*
- Available for C, C++, and Fortran
- You can express parallelism simply with comment-style directives such as

```
#pragma omp parallel
```
- Parallelism can be added incrementally to existing sequential code
- Since results can sometimes match those of serial execution, debugging can be relatively challenging

- OpenMP handles intra-node parallelization (**thread-level parallelism**)
- APIs such as **MPI** handle inter-node parallelization
- Unlike MPI, OpenMP requires no explicit data distribution, simplifying implementation
 - However, achieving optimal performance can be more difficult
 - Simply increasing the thread count is not always sufficient (discussed later)

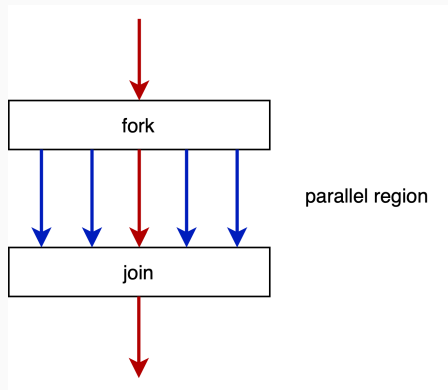
- Since OpenMP 4.0, features for accelerators such as GPUs have been added
- GPU execution is enabled via the `target` directive
- A similar accelerator-focused API is **OpenACC**
 - ACC stands for *Accelerator*
 - Unlike OpenMP, OpenACC is designed specifically for accelerators
 - Enables GPU execution with minimal CUDA code
- Details are omitted here; see the Appendix for a basic OpenMP GPU example

pragma omp parallel

- The most fundamental construct
- All statements within the `#pragma omp parallel` block are executed by each thread
- Note: the same code runs as many times as there are threads
- Threads synchronize at the end of the block (**fork-join model**)

```
1 #pragma omp parallel
2 {
3     // parallel region
4 }
```

Fork-Join Model



- Entering the parallel region spawns multiple threads (**fork**)
- Each thread executes its work in parallel
- Threads synchronize at the end of the region (**join**)

Reference: What is `pragma`

- A *pragma* is a compiler **directive** used to convey instructions to the compiler
- In C/C++, it appears as a line beginning with **`#pragma`**
- In OpenMP, `#pragma omp ...` instructs the compiler to apply parallelization
- Another example is `#pragma once`, used to prevent multiple inclusion of header files

Running `hello.c`

- Execute `hello.c`:
 - Compile with `make`
 - To enable OpenMP in `gcc`, add `-fopenmp` at compile time
 - On macOS, `gcc` may invoke `clang` internally, complicating matters
 - Installing `gcc` via `brew install gcc` and setting `CC := gcc-<ver>` in the Makefile is recommended
- At runtime, set the thread count with:
 - `$ OMP_NUM_THREADS=4 ./hello`
 - Or: `$ export OMP_NUM_THREADS=4`
 - If omitted, `OMP_NUM_THREADS` defaults to the number of logical cores
 - You may also call `omp_set_num_threads(num_threads)` in code

Program of hello.c

- `omp_get_num_threads()` returns the total number of threads
- `omp_get_thread_num()` returns the calling thread's ID

```
1 printf("Before parallel region\n");
2 #pragma omp parallel
3 {
4     int tid = omp_get_thread_num();
5     int nthreads = omp_get_num_threads();
6     printf("Hello, World! I'm thread %d of %d\n", tid, nthreads);
7 }
8 printf("After parallel region\n");
```

Output of hello.c

- Example output:

```
1 $ OMP_NUM_THREADS=4 ./hello
2 Before parallel region
3 Hello, World! I'm thread 1 of 4
4 Hello, World! I'm thread 2 of 4
5 Hello, World! I'm thread 0 of 4
6 Hello, World! I'm thread 3 of 4
7 After parallel region
```

- Each thread executes the code within the parallel block
- The order of lines may vary on each run

- In real-world applications, you typically distribute different work among threads using **Work Sharing constructs**
 - **for construct**: `#pragma omp parallel for` – splits loop iterations
 - **sections construct**: `#pragma omp parallel sections` – executes independent blocks in parallel
 - **task construct**: `#pragma omp task` – dynamically creates tasks for flexible distribution

Contents

1. What is OpenMP?
- 2. Representative Syntax**
3. Loop Scheduling
4. Parallelization Methods Other Than for-loops
 - sections Construct
 - task Construct
5. Variable Attributes
6. Considerations on Thread Count
7. Appendix

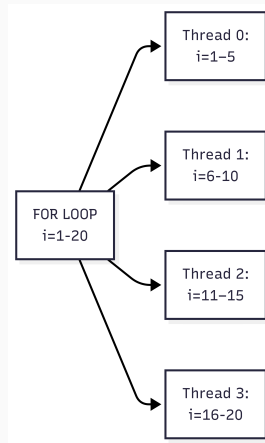
for Construct

- By writing as follows, each iteration of the for loop is distributed across threads

```
1 #pragma omp parallel
2 {
3     #pragma omp for
4     for (int i=0; i<N; i++) {
5         a[i] = i;
6     }
7 }
```

- The same applies if written as follows

```
1 #pragma omp parallel for
2 for (int i=0; i<N; i++) {
3     a[i] = i;
4 }
```



Execution of parallel_for.c

- Let us execute parallel_for.c
- Example output:

```
1 $ OMP_NUM_THREADS=4 ./parallel_for
2 Loop 2 is being processed by thread 1
3 Loop 3 is being processed by thread 1
4 Loop 4 is being processed by thread 2
5 Loop 5 is being processed by thread 2
6 Loop 0 is being processed by thread 0
7 Loop 1 is being processed by thread 0
8 Loop 6 is being processed by thread 3
9 Loop 7 is being processed by thread 3
```

- Starting from the lower indices, loops are assigned to each thread in chunks of $(\text{number of iterations}) / (\text{number of threads})$
 - This assignment method can be modified (described later)

- There are several points to note when parallelizing loops
 1. Shared vs. private variables
 2. Examples that cannot be parallelized
 3. Data races

Caveat 1: Shared and Private Variables

- Variables are classified as **shared** (common to all threads) or **private** (allocated independently for each thread)
 - This is referred to as the variable's **attribute**
- Except for the loop variable immediately following `pragma omp parallel for`, all variables are shared by default (!)
- Thus, a program like the following will not function correctly
 - `j` is a shared variable
 - Between `j = i` and `a[j] = i`, `j` may be overwritten by another thread

```
1 #pragma omp parallel for
2 for (i=0; i<N; i++) {
3     j = i;
4     a[j] = 1;
5 }
```

Caveat 1: Shared and Private Variables

- Observe the incorrect behavior in practice
- Output of `attribution_err.c`:

```
1 $ OMP_NUM_THREADS=4 ./attribution_err
2 Sum = 9986
3 $ OMP_NUM_THREADS=4 ./attribution_err
4 Sum = 9989
5 $ OMP_NUM_THREADS=1 ./attribution_err
6 Sum = 10000
```

- The correct result is `Sum = 10000`

Caveat 1: Shared and Private Variables

- The correct approach is to specify the variable as private using the `private` clause

```
1 #pragma omp parallel for private(j)
2 for (i=0; i<N; i++) {
3     j = i;
4     a[j] = 1;
5 }
```

- Alternatively, using `default(private)` makes private the default attribute

Caveat 1: Shared and Private Variables

- Alternatively, you can also do the following
 - Variables declared inside the parallel region are private by default

```
1 #pragma omp parallel for
2 for (i=0; i<N; i++) {
3     int j = i;
4     a[j] = 1;
5 }
```

- Further details on variable attributes follow later

Caveat 2: Examples That Cannot Be Parallelized

- Not all loops can be parallelized
- Programs where the results differ depending on the order in which i is iterated from 1 to N (flow-dependent) are difficult to parallelize (unless the algorithm is changed)
- Example 1:

```
1 for (i=0; i<N; i++) {  
2     a[i] = a[i] + 1;  
3     b[i] = a[i-1] + a[i+1];  
4 }
```

- $a[i-1]$ may not have been updated

Caveat 2: Examples That Cannot Be Parallelized

- Example 2:

```
1 for (i=0; i<N; i++) {  
2     a[i] = a[b[i]];  
3 }
```

- Whether parallelization is possible depends on the contents of `b[i]`

Caveat 3: Data Races

- As learned in operating systems, handling **data races** is crucial in parallel processing
- For example, the following program will not function correctly

```
1 #pragma omp parallel for
2 for (int i = 0; i < N; i++) {
3     for (int j = 0; j < N; j++) {
4         A[j] += 2.0 * B[i];
5     }
6 }
```

- Another thread may write to `A[j]` between the read and write operations
- OpenMP provides several ways to avoid data races

Avoiding Data Races 1: lock

- OpenMP provides a lock mechanism
- In the previous example, it becomes:

```
1  omp_lock_t lock;
2  omp_init_lock(&lock);
3  #pragma omp parallel for
4  for (int i = 0; i < N; i++) {
5      for (int j = 0; j < N; j++) {
6          omp_set_lock(&lock);
7          A[j] += 2.0 * B[i];
8          omp_unset_lock(&lock);
9      }
10 }
```

Avoiding Data Races 2: critical directive

- Statements inside `pragma omp critical` are executed by at most one thread at a time
- Less flexible than locks, but simpler to implement
- By adding the optional **(name)**, you can create multiple critical sections
 - Sections with the same `name` are mutually exclusive, but different names may execute concurrently
 - The `name` is optional
- In the previous example, it becomes:

```
1 #pragma omp parallel for
2 for (int i = 0; i < N; i++) {
3     for (int j = 0; j < N; j++) {
4         #pragma omp critical
5         {
6             A[j] += 2.0 * B[i];
7         }
8     }
9 }
```

Avoiding Data Races 3: atomic directive

- For a **single simple statement** (e.g. `x+=value` or `x^=value`), `pragma omp atomic` is fast
- Uses a hardware atomic instruction to update the value
- In the previous example, it becomes:

```
1 #pragma omp parallel for
2 for (int i = 0; i < N; i++) {
3     for (int j = 0; j < N; j++) {
4         #pragma omp atomic
5         A[j] += 2.0 * B[i];
6     }
7 }
```

Comparison of lock, critical, and atomic

- Compare the execution times of these methods
- Timing is measured using `omp_get_wtime()`
- Run `lock.c`, `critical.c`, and `atomic.c`
- Example results:

```
1 $ ./lock
2 Elapsed time: 4.004458 seconds
3 $ ./critical
4 Elapsed time: 4.324831 seconds
5 $ ./atomic
6 Elapsed time: 0.070357 seconds
```

Comparison of lock, critical, and atomic

- The above program uses a single lock or critical section, causing unnecessary waiting
- The real contention is on **accesses to the same** $A[j]$; use one lock per element instead
- For example:

```
1  omp_lock_t  locks[N];
2  for (int i = 0; i < N; i++) omp_init_lock(&locks[i]);
3  #pragma omp parallel for
4  for (int i = 0; i < N; i++) {
5      for (int j = 0; j < N; j++) {
6          omp_set_lock(&locks[j]);
7          A[j] += 2.0 * B[i];
8          omp_unset_lock(&locks[j]);
9      }
10 }
```

Comparison of lock, critical, and atomic

- Since the name of critical (probably) cannot be named dynamically, the above implementation is difficult and omitted.
- Example results of `lock2.c`:

```
1 $ ./atomic
2 Elapsed time: 0.070357 seconds
3 $ ./lock2
4 Elapsed time: 0.161204 seconds
```

reduction clause

- For example, used for operations like summing across threads
- Simple example: vector dot product

```
1 // Implementation using the atomic directive
2 #pragma omp parallel for
3 for (int i = 0; i < N; i++) {
4     #pragma omp atomic
5     sum += A[i] * B[i];
6 }
7
8 // Implementation using the reduction clause
9 #pragma omp parallel for reduction(+:sum)
10 for (int i = 0; i < N; i++) {
11     sum += A[i] * B[i];
12 }
```

- Other operations (max, min, etc.) and user-defined reductions are also supported

Implementation of reduction

- Each thread first computes into its own private variable
- Finally, these partial results are combined
- Conceptually:

```
1 #pragma omp parallel
2 {
3     int tid = omp_get_thread_num();
4     #pragma omp for
5     for (int i = 0; i < N; i++) {
6         sum_local[tid] += A[i] * B[i]; // Each thread updates only its own
        sum_local
7     }
8 }
9
10 for (int t = 0; t < nthreads; t++) {
11     sum += sum_local[t]; // Aggregate all partial sums
12 }
```


Comparison of execution times

- Compare three implementations of the sum operation: atomic, reduction, and private-array
- Run `sum_compare.c`
- Example results:

```
1 Elements: 100000, Threads: 4
2 1) reduction      : sum = 200000.000000   time = 0.000297 s
3 2) atomic         : sum = 200000.000000   time = 0.002980 s
4 3) private-array  : sum = 200000.000000   time = 0.000370 s
```

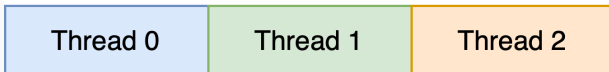
1. What is OpenMP?
2. Representative Syntax
- 3. Loop Scheduling**
4. Parallelization Methods Other Than for-loops
 - sections Construct
 - task Construct
5. Variable Attributes
6. Considerations on Thread Count
7. Appendix

- Clearly, scheduling is crucial for performance optimization
- Even with parallelization, uneven load distribution degrades performance
- You must wait until the thread with the heaviest workload finishes

- There are three types of scheduling for for-loops
 1. `static`: statically assign chunks of iterations per thread
 2. `dynamic`: dynamically assign chunks; threads that finish take the next chunk
 3. `guided`: assign chunks of gradually decreasing size to threads as they finish
- Specify with `#pragma omp parallel for` `schedule(dynamic, chunk)` (default is `static` if omitted)

- `schedule(static, chunk)`
 - Divide the loop into chunks of size `chunk` and assign them statically in round-robin starting from thread 0
 - If the schedule clause is omitted, the default is `static` with chunk size = (number of iterations) / (number of threads) (see `02_syntax/parallel_for.c`)

$\text{chunk} = (\text{iter_num}) / (\text{threads_num})$



$\text{chunk} = 1$



- `schedule(dynamic, chunk)`
 - Divide the loop into chunks of size `chunk`; as each thread finishes, it requests the next chunk
 - Effective when the workload per iteration is not known until runtime
 - If `chunk` is omitted, it defaults to 1



- `schedule(guided, chunk)`
 - Divide the loop into chunks, assigning them with **gradually decreasing chunk sizes** to threads as they finish
 - `chunk` is the minimum chunk size
 - If you specify a `chunk > 1`, the chunk size decreases exponentially down to `chunk`, though the final chunk may be smaller than `chunk`
 - If `chunk` is omitted, it defaults to 1



Characteristics of Each Scheduling

- For `dynamic` and `guided`, the chunk size affects performance
 - Too small a chunk size improves load balance but increases scheduling overhead
 - Too large a chunk size reduces overhead but worsens load balance
 - Tuning the chunk size is important
- Scheduling overhead:
 - `dynamic` and `guided` have higher overhead than `static`, since scheduling occurs at runtime
 - **If you can arrange a statically balanced distribution in advance**, `static` scheduling may yield even higher performance

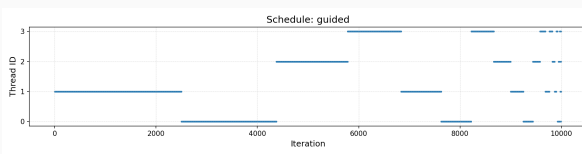
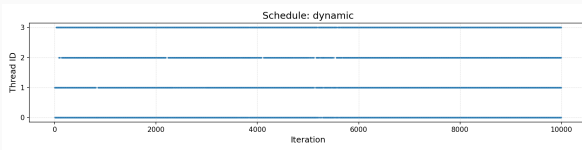
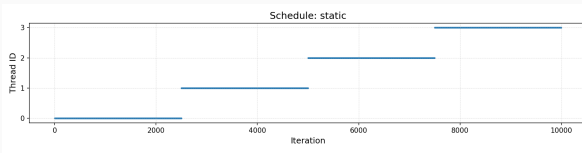
Comparison of Scheduling

- Parallelize only the outer loop of the following code (later iterations do more work)
- Test the three scheduling patterns above and compare execution times
- Also record which loop iteration each thread handles

```
1 for (int i = 0; i < N; i++) {  
2     for (int j = 0; j < i; j++) {  
3         A[i]++;  
4     }  
5 }
```

Comparison of Scheduling: Results

- Run `static.c`, `dynamic.c`, and `guided.c` with 4 threads



Comparison of Scheduling: Measured Times

- Example output:

```
1 === OpenMP threads: 4 ===  
2  static : 0.061116 s  
3  dynamic : 0.052824 s  
4  guided : 0.043387 s
```

- Although there was variability in timing:
- `static` is the slowest
 - Load imbalance occurs because threads handling later iterations take longer
- `guided` is faster than `dynamic`
 - Early coarse assignment reduces scheduling overhead compared to `dynamic`

Changing static Chunk Size

- Setting the chunk size to 1 yields a more even division (note the original default chunk size is $(\text{iterations}) / (\text{threads})$)
- Run `static` with `chunk=1`

```
1 === OpenMP threads: 4 ===  
2  static : 0.045067 s  
3  dynamic : 0.052824 s  
4  guided  : 0.043387 s
```

- Although I might expect `static` to outperform `guided`, it did not clearly do so
 - Possible cache-hit differences?
guided sees contiguous accesses early, whereas `static` strides by 4
 - The order of scheduling overhead is not well understood

Reference: collapse Clause

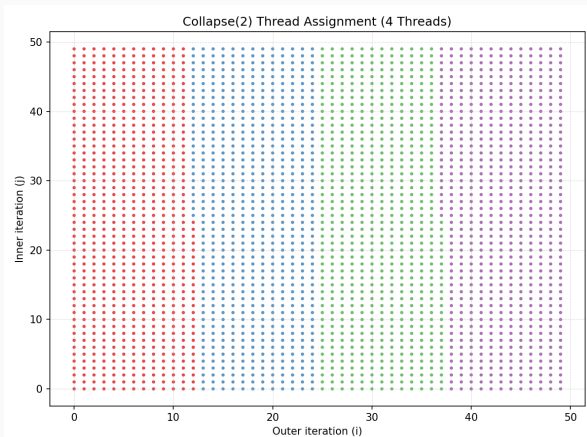
- When parallelizing nested loops, you can use the `collapse` clause

```
1 #pragma omp parallel for collapse(2)
2 for (int i = 0; i < N; i++) {
3     for (int j = 0; j < N; j++) {
4         A[i] += B[j];
5     }
6 }
```

- You cannot insert other operations between the loops (must be a pure nested loop)
- `collapse(n)` treats n loops as a **single large loop** for parallelization
- Can be combined with the `schedule` clause

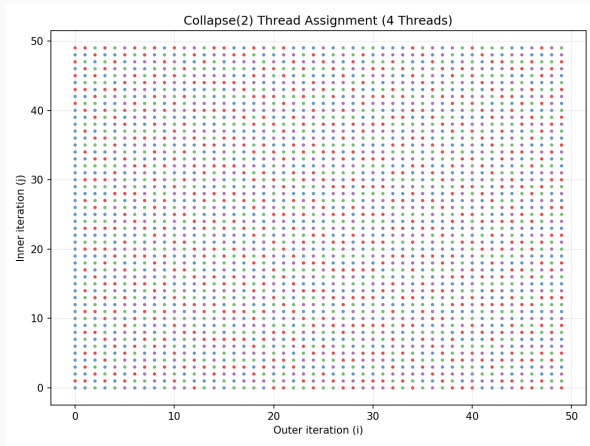
Examining collapse Behavior

- Run `collapse.c`
- Visualize which iterations each thread handles, color-coded
- Execution with no `schedule` specified (defaults to `static`):



Examining collapse Behavior

- Execution with `schedule(dynamic, 1)`:



When collapse Is Useful

- Expanding nested loops increases the number of iterations available for distribution
 - Might not bring much benefit for CPU programs
- Useful, for example, in GPU parallelization of loops
- See `07_appendix/matmul_gpu.c` for details

Contents

1. What is OpenMP?
2. Representative Syntax
3. Loop Scheduling
4. Parallelization Methods Other Than for-loops
 - sections Construct
 - task Construct
5. Variable Attributes
6. Considerations on Thread Count
7. Appendix

Parallelization Beyond Loops 1: sections Construct

- A more flexible(?) parallelization method than the for construct
- Each section is assigned to a thread

```
1 #pragma omp parallel sections
2 {
3     #pragma omp section
4     {
5         work1();
6     }
7     #pragma omp section
8     {
9         work2();
10    }
11 }
```

- Details omitted

Contents

1. What is OpenMP?
2. Representative Syntax
3. Loop Scheduling
4. Parallelization Methods Other Than for-loops
 - sections Construct
 - task Construct**
5. Variable Attributes
6. Considerations on Thread Count
7. Appendix

Beforehand: single Construct

- The body of a single construct is executed by only one thread
- Which thread executes it is unspecified
- Unless you use the `nowait` clause, **there is a barrier after single**
- See `single.c` for details

```
1 #pragma omp parallel
2 {
3     #pragma omp single
4     {
5         work();
6     }
7 }
```

Beforehand: master Construct

- Similar to single
- However, master is **always executed by the master thread**
- There is **no barrier** after master
- See `master.c` for details

```
1 #pragma omp parallel
2 {
3     #pragma omp master
4     {
5         work();
6     }
7 }
```

Parallelization Beyond Loops 2: task Construct

- Suited to parallelizing dynamically and irregularly generated work
- Create a task with `#pragma omp task`
- Wait for tasks to complete with `#pragma omp taskwait`

Parallelization Beyond Loops 2: task Construct

- Example: computing Fibonacci numbers

```
1  int fib(int n) {
2      if (n < 2) return n;
3      else {
4          #pragma omp task shared(x) firstprivate(n)
5          x = fib(n - 1);
6          #pragma omp task shared(y) firstprivate(n)
7          y = fib(n - 2);
8          #pragma omp taskwait
9          return x + y;
10     }
11 }
12
13 #pragma omp parallel
14 {
15     #pragma omp master
16     printf("Fibonacci of %d is %d\n", 10, fib(10));
17 }
```

Execution of fib.c

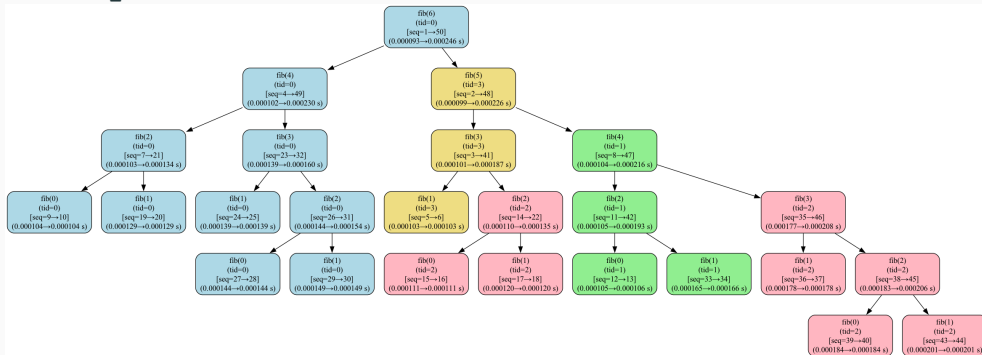
- Run fib.c
- Example output:

```
1 $ ./fib
2 serial version
3 Fibonacci of 50 is 12586269025
4 Elapsed time: 51.424394 seconds
5
6 $ OMP_NUM_THREADS=4 ./fib
7 OpenMP version
8 threads: 4
9 Fibonacci of 50 is 12586269025
10 Elapsed time: 23.339175 seconds
```

- There is overhead for task creation and management
 - For $n = 50$, about 2^{50} tasks!?
 - Avoid by using serial execution when n is small

Visualizing Task Execution

- Run `fib_tree.c` to visualize task execution



- [Link to GIF](#)

Contents

1. What is OpenMP?
2. Representative Syntax
3. Loop Scheduling
4. Parallelization Methods Other Than for-loops
 - sections Construct
 - task Construct
- 5. Variable Attributes**
6. Considerations on Thread Count
7. Appendix

- **private**: Each thread gets its own storage; **initial value at region entry is undefined**
- **shared**: All threads refer to the same variable; concurrent access requires synchronization or undefined behavior
- **firstprivate**: Like private, but **initialized to the parent thread's value** at region entry
- **lastprivate**: After the region, the value from the last iteration is copied back to the parent thread
- See `attribution.c` for details

Contents

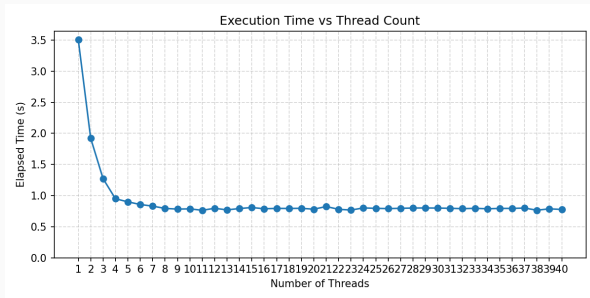
1. What is OpenMP?
2. Representative Syntax
3. Loop Scheduling
4. Parallelization Methods Other Than for-loops
 - sections Construct
 - task Construct
5. Variable Attributes
6. Considerations on Thread Count
7. Appendix

What Is the Optimal Number of Threads?

- Increasing the number of threads does not always improve performance
- Threads can only execute concurrently up to the number of **logical cores**
- Commands to determine available logical cores:
 - Linux: `nproc`
 - Mac: `sysctl -n hw.logicalcpu`
- Or run `max_threads.c`
 - `omp_get_max_threads()` returns the maximum number of threads available
- On my Mac, the logical core count is 8

Relationship Between Thread Count and Execution Time (1)

- Parallel matrix multiplication program `measure_threads_1.c` was run varying the thread count
- Example results:



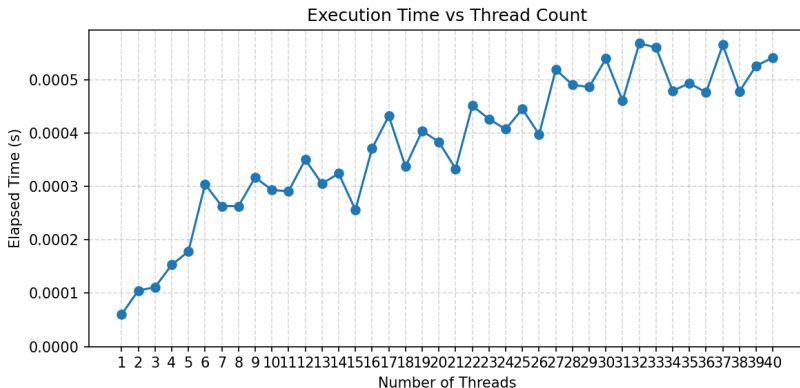
- Performance plateaus around 8 threads
 - Indeed, the limit corresponds to the number of logical cores

Relationship Between Thread Count and Execution Time (2)

- In some cases, adding threads can degrade performance
 - For example, in programs with mutual exclusion, when a thread holding a lock is preempted by the OS (**preemption**), it incurs time loss until it resumes
- Simple counter-increment program `measure_threads_2.c` was run varying the thread count

Relationship Between Thread Count and Execution Time (2)

- Example results:



- Performance worsens as the number of threads increases
- **Whether parallelization yields benefit depends on the program structure**

Contents

1. What is OpenMP?
2. Representative Syntax
3. Loop Scheduling
4. Parallelization Methods Other Than for-loops
 - sections Construct
 - task Construct
5. Variable Attributes
6. Considerations on Thread Count
7. Appendix

1. SIMD Support in OpenMP
2. Examples of Programs
3. Implementation of Various Directives

- SIMD allows executing the same instruction on multiple data elements in parallel
- In OpenMP, use `#pragma omp simd` to enable SIMD

```
1 #pragma omp simd
2 for (int i = 0; i < N; i++) {
3     b[i] = 2 * a[i];
4 }
```

Reference: Compilation and Optimization Report

- Compile with the following to output vectorization reports to `vec_report.txt`
- `-O3` specifies the optimization level

```
1 $ gcc-15 -O3 -fopt-info-vec-optimized=vec_report.txt -fopenmp simd.c -o simd
```

- Example output:

```
1 simd.c:30:21: optimized: loop vectorized using 16 byte vectors
2 simd.c:30:21: optimized: loop vectorized using 8 byte vectors
3 simd.c:18:10: optimized: loop vectorized using 16 byte vectors
```

- Confirms that loops were vectorized

Reference: Generating Assembly

- Compile with the following to output assembly to `simd.s`

```
1 $ gcc-15 -O1 -fopenmp -S simd.c -o simd.s
```

- Example output:

```
1 ldr q31, [x7, x1]           // Load 128 bits (4 ints) from x7 + x1
2 add v31.4s, v31.4s, v31.4s  // Double each of the four 32-bit ints in v31
3 str q31, [x0, x1]           // Store 128 bits to x0 + x1
```

- After loading into vector registers, four integers are doubled at once
- On Mac, ARM Neon SIMD extensions are used
- With 128-bit vector width, can process four 32-bit ints or two 64-bit floats in one instruction

Execution Time Comparison With and Without SIMD

- Run `simd.c`
- With SIMD enabled (compiled at `-O1` with `#pragma omp simd`):

```
1 $ OMP_NUM_THREADS=1 ./simd
2 Dot product (SIMD): 0.161255 sec
3 $ OMP_NUM_THREADS=4 ./simd
4 Dot product (SIMD): 0.151914 sec
```

- Without SIMD (compiled at `-O1` without `#pragma omp simd`):

```
1 $ OMP_NUM_THREADS=1 ./simd
2 Dot product (scalar): 0.215824 sec
3 $ OMP_NUM_THREADS=4 ./simd
4 Dot product (scalar): 0.151753 sec
```

Execution Time Comparison With and Without SIMD

- Note measurement variability
- SIMD benefit is higher with fewer threads (but not a $4\times$ speedup)
 - Memory access becomes the bottleneck as thread count increases
- On this setup, compiling at `-O3` enabled SIMD automatically, even without `#pragma omp simd`

Example of for-parallel Program: Monte Carlo Method

- Approximating π using the Monte Carlo Method
 - Generate random points and compute π from the ratio of points inside the circle
 - Each thread independently generates points and counts those inside the circle
 - Finally, combine all thread counts with a `reduction` to compute π
- See `monte_carlo.c`

Example of task-parallel Program: Merge Sort

- Parallelize recursive merge sort using OpenMP `task`
 - Split the array into two and register each sort as a task
 - When subarrays become sufficiently small, sort sequentially (e.g. insertion sort)
 - Wait for parallel tasks with `taskwait`, then merge
 - Enables recursive decomposition and parallelization, which is difficult with parallel `for`
- See `parallel_merge_sort.c`

Example of GPU Execution Program: Matrix Multiplication

- Implement GPU parallelization of matrix multiplication using OpenMP
- See `matmul_gpu.c`

```
1 #pragma omp target data map(to:A[0:N][0:N], B[0:N][0:N]) map(from:C[0:N][0:N])
   )
2 {
3     #pragma omp target teams distribute parallel for collapse(2)
4     for(int i = 0; i < N; i++) {
5         for(int j = 0; j < N; j++) {
6             for(int k = 0; k < N; k++) {
7                 C[i][j] += A[i][k] * B[k][j];
8             }
9         }
10    }
11 }
```

Example of GPU Execution Program: Matrix Multiplication

- Here, compare execution time with and without `collapse(2)` (run on miyabi-g)
- Example output:

```
1 $ ./matmul_gpu
2 Use collapse
3 Test passed!
4 Computation time: 0.386624 seconds
5
6 $ ./matmul_gpu
7 No collapse
8 Test passed!
9 Computation time: 0.702773 seconds
```

- `collapse` increases the iteration count from N to N^2 , distributing work across more GPU threads

Implementation of Various Directives

- I have not examined every detail, so please forgive any mistakes
- We will discuss only the internal implementation of the OpenMP runtime in gcc (libgomp)
- The implementation is shown at
<https://github.com/gcc-mirror/gcc/tree/releases/gcc-9.2.0/libgomp>

Implementation of Various Directives 1: Thread Creation

- The `parallel` directive expands as:

```
1 void subfunction (void *data)
2 {
3     use data;
4     body;
5 }
6
7 setup data;
8 GOMP_parallel_start (subfunction, &data, num_threads);
9 subfunction (&data);
10 GOMP_parallel_end ();
```

- `GOMP_parallel_start` calls `gomp_team_start()`, which spawns OS-level threads (Linux, Mac: `pthread_create`)
- See `parallel.c` and `team.c` in the git repository

Implementation of Various Directives 2: lock Implementation

- Equivalent to `pthread_mutex` as far as I have found
- Defined in `config/posix/mutex.h` as:

```
1 typedef pthread_mutex_t gomp_mutex_t;  
2 static inline void gomp_mutex_init (gomp_mutex_t *mutex)  
3 {  
4     pthread_mutex_init (mutex, NULL);  
5 }  
6 static inline void gomp_mutex_lock (gomp_mutex_t *mutex)  
7 {  
8     pthread_mutex_lock (mutex);  
9 }  
10 // ...
```

- See `lock.c`

Implementation of Various Directives 3: critical Implementation

- Definition of an unnamed critical section:

```
1 void
2 GOMP_critical_start (void)
3 {
4     /* There is an implicit flush on entry to a critical region. */
5     __atomic_thread_fence (MEMMODEL_RELEASE);
6     gomp_mutex_lock (&default_lock);
7 }
8
9 void
10 GOMP_critical_end (void)
11 {
12     gomp_mutex_unlock (&default_lock);
13 }
```

- I assume ultimately managed by a mutex
- See `critical.c`

Implementation of Various Directives 4: atomic Implementation

- Compiling 07_appendix/atomic1.c to assembly yields:

```
1 ldadd w1, w0, [x0]
```

- On ARM, `ldadd` atomically loads from `[x0]`, adds `w0`, stores the result back to `[x0]`, and returns the original memory value in `w1`

Implementation of Various Directives 4: atomic Implementation

- What happens if it is slightly more complex?
- Compiling 07_appendix/atomic2.c to assembly yields:

```
1 LFB0:
2     fadd    d0, d0, d0        // b = 2 * b
3     ldr     x1, [x0]          // load *a into x1
4 L2:
5     fmov    d31, x1           // move x1 into d31
6     mov     x2, x1            // copy x1 to x2
7     fadd    d31, d0, d31      // d31 = *a + 2*b
8     fmov    x3, d31           // store new value in x3
9     cas     x2, x3, [x0]      // compare [x0] with x2 and, if equal, store x3;
                                // returns old value in x2
10    cmp     x1, x2            // if x1 == x2, success
11    bne     L3                // retry if failed
12    ret
```

- Implemented as a CAS loop

References

- http://www.den.t.u-tokyo.ac.jp/ad_prog/parallel/
- https://www.hpci-office.jp/documents/HPC_Programming_Seminar/openmp-part2_20240417.pdf
- <https://theartofhpc.com/pcse/index.html>
- <https://www.openmp.org>
- <https://gcc.gnu.org/onlinedocs/libgomp/>
- <https://developer.arm.com/documentation/dht0002/a/Introducing-NEON?lang=en>
- Katagiri Takahiro, “Introduction to Parallel Programming: Learning OpenMP and OpenACC through Sample Programs”
- The University of Tokyo Supercomputer Programming Lecture Materials